Acrobot AI Agent using Deep Reinforcement Learning

- *Deep Q-Network (DQN) Implementation on Acrobot-v1 Environment*

Team Member:

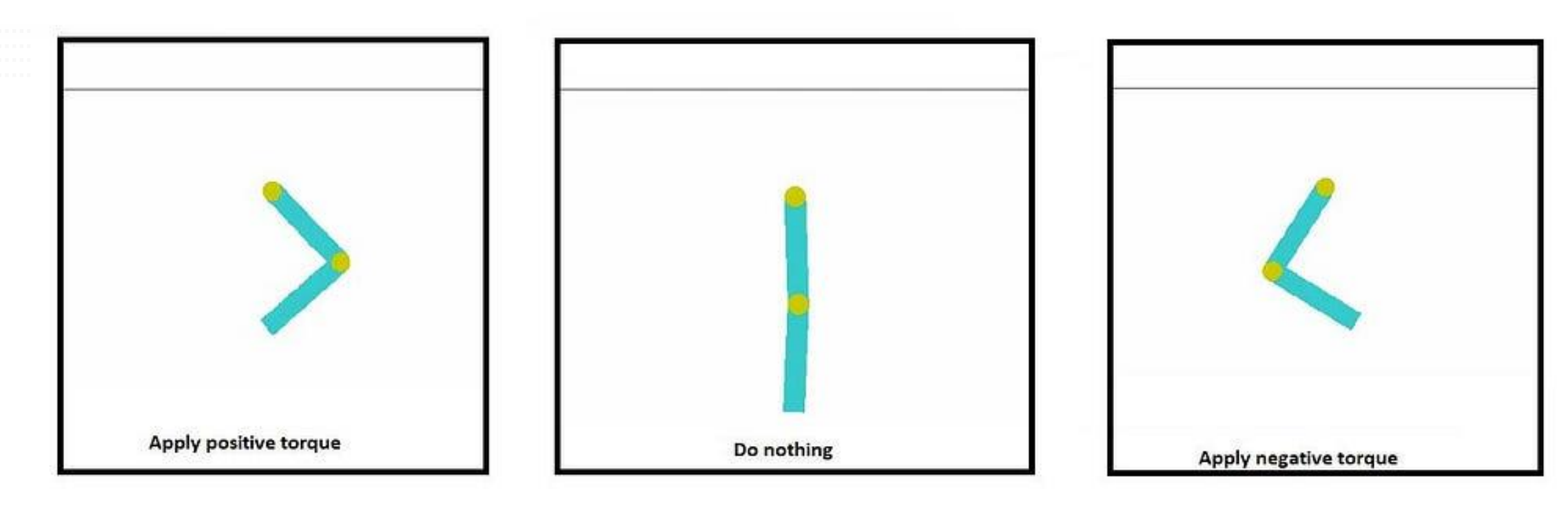
1-Maryam Ashraf Attya

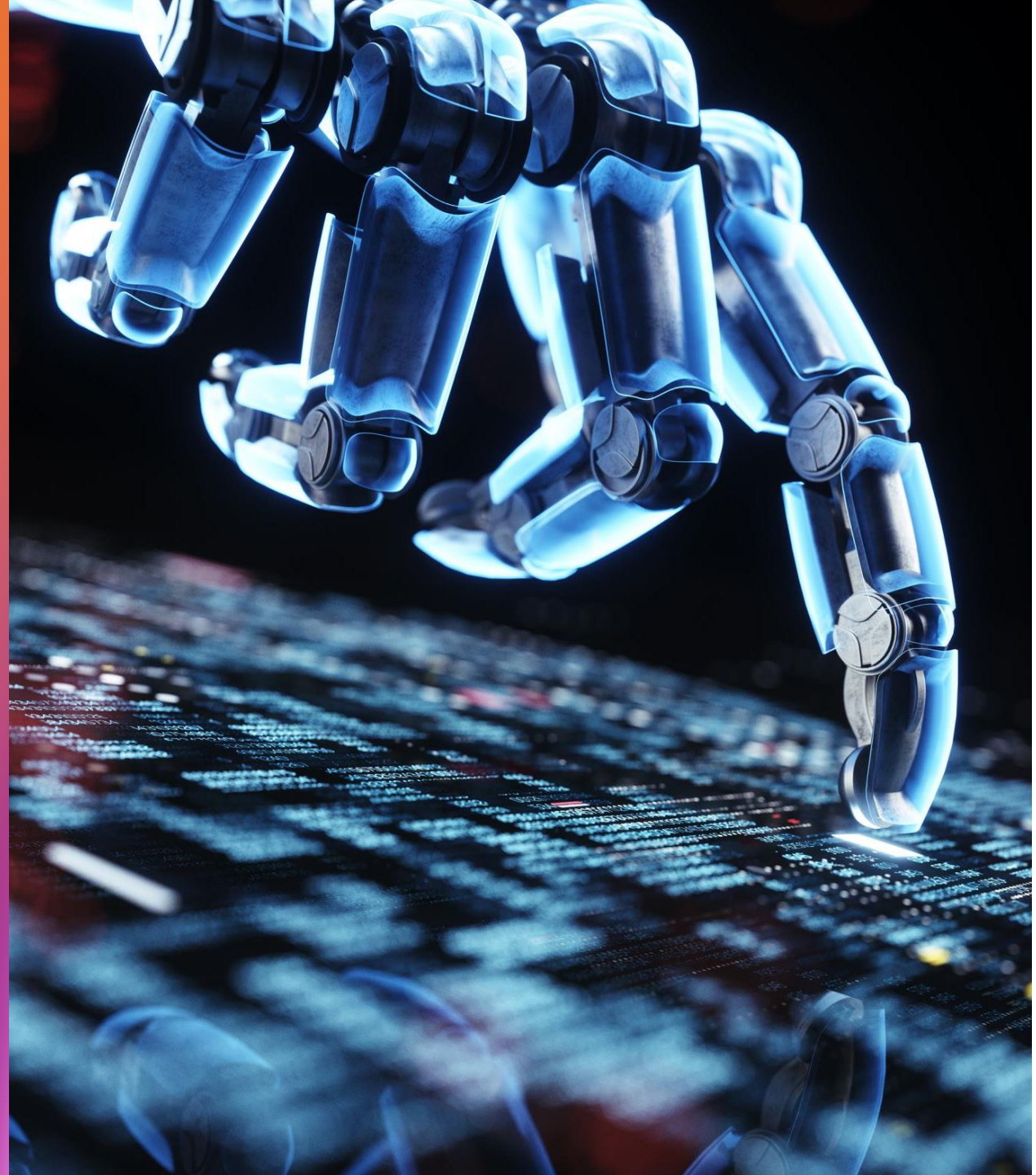
2-Amira Emad Selim

3-Rahma Mohamed Sophy

Acrobot-v1 Environment

- A simulated double-link robotic arm
- Goal: swing the lower link above a target height
- Physics-based environment from Gymnasium





Problem Statement

- Key Points:
- Control a 2-link robotic arm
- Apply torque to joints
- Use physics to reach target height
- Objective:
- “The agent must swing the robotic arm upward using momentum and torque control.”

```
ckpt = load_from_hub(
    repo_id="sb3/dqn-Acrobot-v1",
    filename="dqn-Acrobot-v1.zip",
)

env = gym.make("Acrobot-v1")
```

Environment Overview

- Action Space: 3 discrete actions (apply torque)
- Observation Space: 6 continuous values
- Reward: -1 per step until goal is reached
- Episode ends when goal is achieved or max steps reached

DQN Network Architecture

- **Policy Type:**
- DQNPolicy (Deep Q-Network)

Network Structure:

- **Input Layer (State Vector - 6 values):**
- cos(θ1)
- sin(θ1)
- cos(θ2)
- sin(θ2)
- angular velocity 1
- angular velocity 2

Hidden Layers:

- Dense Layer → 256 neurons + ReLU
- Dense Layer → 256 neurons + ReLU

Output Layer:

- 3 Q-values representing actions:
  - Torque Left ⬅️
  - No Torque ⏸️
  - Torque Right ➡️

```
[Experience Replay]
  buffer_size                = 1
  batch_size                 = 128
  learning_starts            = 0
  train_freq                 = TrainFreq(frequency=4, unit=<TrainFrequencyUnit.STEP: 'step'>)
  gradient_steps             = -1

[Target Network]
  target_update_interval     = 250

[Exploration]
  exploration_fraction       = 0.12
  exploration_final_eps      = 0.1
  exploration_rate           = 0.1

[Core RL]
  gamma                      = 0.99
  learning_rate              = 0.0001
```

Experience replay helps the model learn from past experiences, while the target network stabilizes training. Exploration settings control how the agent balances exploration and exploitation, and gamma defines how much future rewards are considered

DQN Hyperparameters (Acrobot-v1)

Pretrained Model Performance

Pretrained Agent Evaluation

- The pretrained DQN model is tested before fine-tuning
- The agent interacts with the Acrobot environment
- Performance is measured using total episode reward

📉 Result (Before Fine-Tuning)

- The model shows unstable swing-up behavior
- Average reward is relatively low (around -70 to -80)
- The agent fails to consistently reach the goal

🎥 Video Recording

- Episodes are recorded using RGB rendering
- Simulation is saved as MP4 video for analysis

```python
env = gym.make("Acrobot-v1", render_mode="rgb_array")


all_frames = []
num_episodes = 5

print(f"Recording {num_episodes} episodes of Acrobot...")

for i in range(num_episodes):

    obs, info = env.reset()
    done = False
    score = 0

    while not done:
        action, _ = model.predict(obs, deterministic=True)

        obs, reward, terminated, truncated, info = env.step(action)

        done = terminated or truncated
        score += reward

        frame = env.render()
        all_frames.append(frame)
```

Fine-Tuning Idea

- Instead of training from scratch, we start from a pretrained DQN model
- The goal is to improve existing knowledge, not learn from zero
- Fine-tuning makes training faster and more stable
- We improve performance using new experiences from Acrobot

```
ft_model = DQN.load(ckpt, env=ft_env, custom_objects=custom_objects)


REPLAY_BUFFER_SIZE = 20000
ft_model.buffer_size = REPLAY_BUFFER_SIZE
ft_model.replay_buffer = ft_model.replay_buffer.__class__(
    REPLAY_BUFFER_SIZE,
    ft_model.observation_space,
    ft_model.action_space,
    device=ft_model.device,
    optimize_memory_usage=ft_model.optimize_memory_usage,
)


ft_model.exploration_rate = 0.02
ft_model.exploration_schedule = get_linear_fn(0.02, 0.02, 1.0)
```

```
callback = EarlyStoppingBestWeights(
    eval_env          = ft_env,
    eval_freq         = 1500,
    n_eval_episodes   = 10,
    patience          = 15,

    min_reward        = max(mean_r + 5, -80),
    floor_patience    = 4,
)
```

Early Stopping Callback

- Monitors model performance during training
- Saves the best performing model automatically
- Stops training if no improvement for several evaluations
- Helps prevent overfitting and wasted training time

```python
ft_model.learning_starts = 10_000_000

ft_model.learn(
    total_timesteps=REPLAY_BUFFER_SIZE,
    reset_num_timesteps=False,
    log_interval=100,
)
```

Phase 1: Buffer Pre-population

- First step is collecting experiences from the environment
- These experiences are stored in the replay buffer
- The model does NOT learn yet in this phase
- This helps stabilize future training

```
FINE_TUNE_STEPS = 50_000


finetune_lr = ft_model.policy.optimizer.param_groups[0]['lr'] / 10
ft_model.lr_schedule = get_linear_fn(finetune_lr, finetune_lr, 1.0)
ft_model.gradient_steps = 1
ft_model.learning_starts = 0


callback = EarlyStoppingBestWeights(
    eval_env         = ft_env,
    eval_freq        = 1500,
    n_eval_episodes  = 10,
    patience         = 15,

    min_reward       = max(mean_r + 5, -80),
    floor_patience   = 4,
)

ft_model.learn(total_timesteps=FINE_TUNE_STEPS, reset_num_timesteps=False, log_interval=100, callback=callback)

callback.restore_best_weights()
```

- Model starts learning from collected experiences
- Learning rate is reduced for stability
- Callback is used for monitoring and early stopping
- Best weights are restored after training

Phase 2: Fine-Tuning

```
baseline_rewards, baseline_lengths = evaluate_model(
    baseline_model,
    n_episodes=20
)
# FINE-TUNED EVALUATION

print("\n================ FINE-TUNED MODEL ================")

ft_rewards, ft_lengths = evaluate_model(
    fine_tuned_model,
    n_episodes=20
)
```

- Evaluation Process
- Compared pretrained and fine-tuned DQN models
- Evaluated on 20 episodes using deterministic actions
- Measured:
  - Average Reward
  - Average Steps
  - Reward Stability

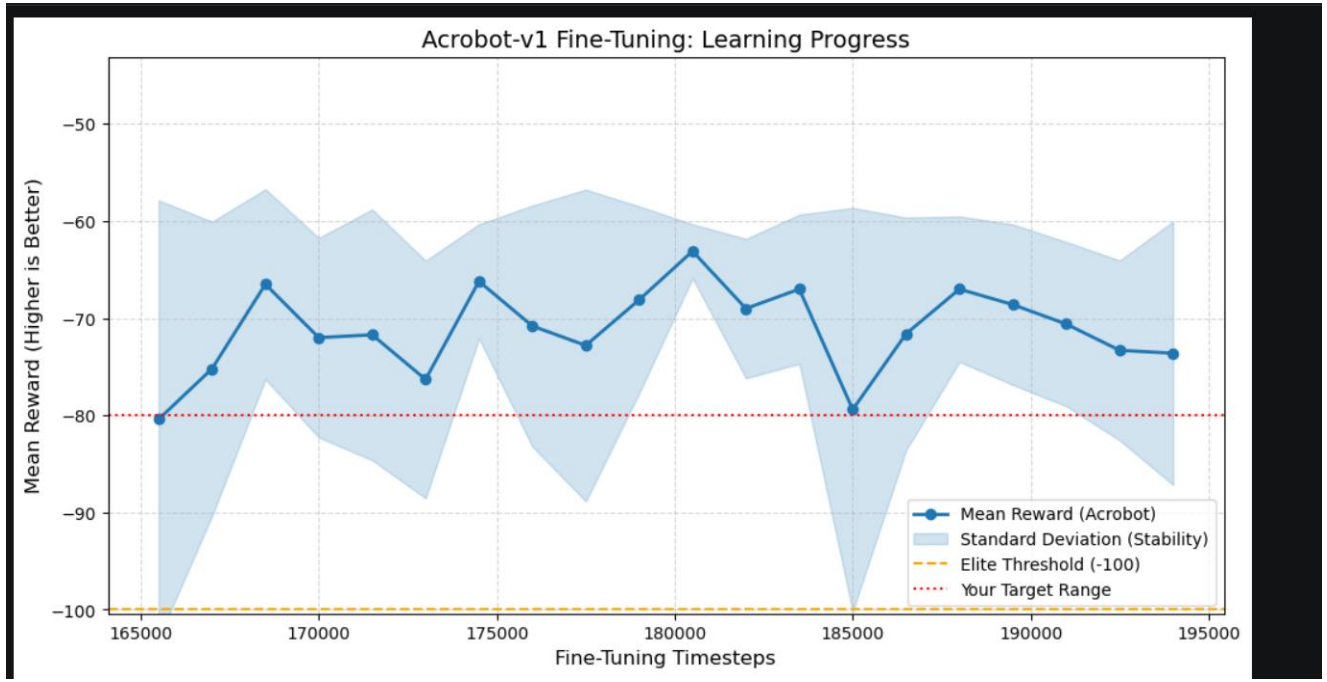
Model Evaluation

Final Results & Comparison

```python
print("FINAL COMPARISON")

baseline_avg = np.mean(baseline_rewards)
ft_avg = np.mean(ft_rewards)

print(f"Baseline Avg Reward    : {baseline_avg:.2f}")
print(f"Fine-tuned Avg Reward  : {ft_avg:.2f}")

print(f"\nBaseline Avg Steps     : {np.mean(baseline_lengths):.2f}")
print(f"Fine-tuned Avg Steps   : {np.mean(ft_lengths):.2f}")

reward_improvement = ft_avg - baseline_avg

print(f"\nReward Improvement: {reward_improvement:.2f}")

if baseline_avg != 0:
    print(f"Improvement %: {(reward_improvement / abs(baseline_avg)) * 100:.2f}%")

print("\nRESULT:")
```

```
------------------------------------
FINAL COMPARISON
====================================
Baseline Avg Reward    : -76.90
Fine-tuned Avg Reward  : -70.45

Baseline Avg Steps     : 77.90
Fine-tuned Avg Steps   : 71.45

Reward Improvement: 6.45
Improvement %: 8.39%
```

- Fine-tuned model achieved higher rewards
- Required fewer steps to complete the task
- Performance improved by approximately 8.4%



- The graph shows the learning progress during fine-tuning
- Mean reward improves over time
- Shaded region represents reward stability
- The model gradually approaches the target performance range

Learning Curve & Training Progress